CME-CAD: Heterogeneous Collaborative Multi-Expert Reinforcement Learning for CAD Code Generation

Ke Niu[1]†, Haiyang Yu[1,2]†*, Zhuofan Chen[1]†, Zhengtao Yao[2], Weitao Jia[2]
Xiaodong Ge[1], Jingqun Tang[2], Benlei Cui[2], Bin Li[1]*, Xiangyang Xue[1] * [1]Fudan University, Shanghai, China.
[2]ByteDance Inc.

{kniu22, zfchen23, xdge25}@m.fudan.edu.cn, zyao9248@usc.edu

{hyyu20, libin, xyxue}@fudan.edu.cn, tangjingqun@bytedance.com

Abstract

Computer-Aided Design (CAD) is essential in industrial design, but the complexity of traditional CAD modeling and workflows presents significant challenges for automating the generation of high-precision, editable CAD models. Existing methods that reconstruct 3D models from sketches often produce non-editable and approximate models that fall short of meeting the stringent requirements for precision and editability in industrial design. Moreover, the reliance on text or image-based inputs often requires significant manual annotation, limiting their scalability and applicability in industrial settings. To overcome these challenges, we propose the Heterogeneous Collaborative Multi-Expert Reinforcement Learning (CME-CAD) paradigm, a novel training paradigm for CAD code generation. Our approach integrates the complementary strengths of these models, facilitating collaborative learning and improving the model's ability to generate accurate, constraint-compatible, and fully editable CAD models. We introduce a two-stage training process: Multi-Expert Fine-Tuning (MEFT), and MultiExpert Reinforcement Learning (MERL). Additionally, we present CADExpert, an open-source benchmark consisting of 17,299 instances, including orthographic projections with precise dimension annotations, expert-generated Chain-of-Thought (CoT) processes, executable CADQuery code, and rendered 3D models. The CADExpert dataset is publicly available at https://modelscope.cn/ datasets/zhuofanChen/CADExpert.

1. Introduction

As a fundamental tool in industrial design, Computer-Aided Design (CAD) has long been essential for the digital creation of complex engineering systems through its formalized modeling language. In modern smart manufacturing, the "digital-first" paradigm governs product development, where design concepts are translated into high-fidelity CAD models prior to physical production. However, this process is heavily reliant on engineers' specialized skills, which are honed through years of training. As a result, automating the generation of precise, constraint-compatible, and fully editable CAD models remains a significant challenge, and represents a critical area of focus for both industry and academia in the advancement of intelligent design.

*Corresponding author: Haiyang Yu (hyyu20@fudan.edu.cn). †Equal contribution.

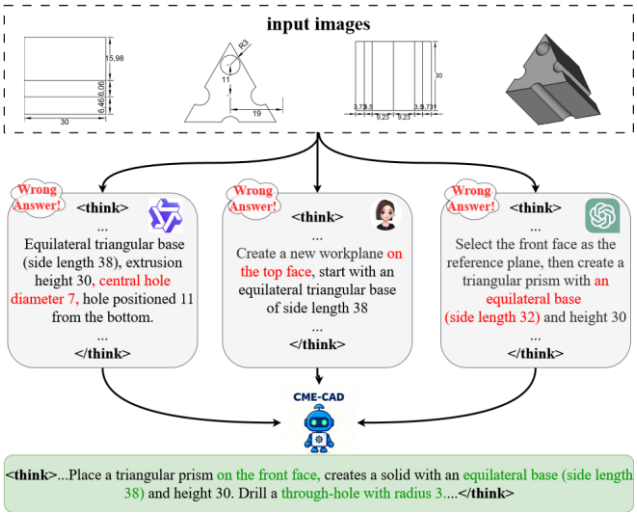

Figure 1. The workflow of our method for CAD code generation.

【2】However, existing approaches to CAD code generation face two primary challenges. From the perspective of industrial design workflows, the current paradigm of generating CAD code from text and image inputs is both costly and dif-ficult to implement in practical applications. This challenge arises primarily due to the necessity of manually annotating textual descriptions, which requires significant expertise. In contrast, 2D engineering drawings are more readily accessible and inherently contain the necessary information, based on design specifications, to construct accurate 3D models. From a methodological standpoint, existing approaches predominantly rely on reinforcement learning with verifiable rewards (RLVR). However, because RLVR is an on-policy method, performance improvements are primarily achieved by optimizing the model along pre-existing, reward-rich reasoning paths. In essence, RLVR refines the model within its current knowledge base, rather than actively exploring novel information. When the reasoning paths are biased the model's ability is inherently restricted. Furthermore, in complex reasoning scenarios, the model's initial strategy often fails to generate correct outputs, making the optimization process difficult. Therefore, it is essential to encourage the generation of more diverse responses within logically consistent reasoning paths, thereby increasing the likelihood of obtaining correct rewards.

【3】To tackle these challenges, we focus on addressing two key aspects. First, from the perspective of industrial design workflows, we investigate the generation of precise CADQuery code directly from orthographic projection inputs, aligning more closely with real-world design practices. Second, from a methodological standpoint, inspired by the principle of "learning from the strengths of others" we introduce the Heterogeneous Collaborative Multi-Expert Reinforcement Learning (CME-CAD) paradigm, specifically designed for generating executable, high-precision CAD code. The workflow of our method is shown in Fig. 1. Central to this approach is the integration of multiple heterogeneous pre-trained models, which leverage their complementary strengths to facilitate collaborative learning. This collaborative learning process enhances the model's ability to generate accurate outputs, providing more reliable learning signals throughout training. Importantly, rather than simply aggregating the outputs of these models, we enable them to learn from one another's strengths, fostering collaboration and driving the model's overall performance to new heights. The training process consists of two stages: 【4】

- Multi-Expert Fine-Tuning (MEFT) focuses on generating reasoning paths with distinct styles by leveraging the unique capabilities of multiple heterogeneous experts. This stage refines each expert's individual reasoning pattern, enhancing their contributions to the overall model.
- Multi-Expert Reinforcement Learning (MERL) promotes cross-expert learning by enabling knowledge transfer between experts. Additionally, it incorporates the hard negative sample buffering mechanism, ensuring the model continues to learn from difficult cases, mitigating reward sparsity, and significantly improving performance.

Another critical bottleneck in CAD code generation is the lack of high-quality open-source datasets that meet industrial-grade requirements. To address this, we introduce CADExpert, a benchmark specifically crafted for the generation of executable and editable CAD code. We have carefully designed a professional and robust method for dataset construction, ensuring that it aligns with real-world industrial needs. CADExpert contains 17,299 instances. Each sample consists of four components: (1) orthographic projections with precise dimensional annotations, (2) detailed code generation processes produced by various expert models, (3) the corresponding executable CADQuery code, and (4) the 3D CAD model. This comprehensive dataset is designed to support robust training and evaluation of CAD code generation methods, providing a diverse and high-quality resource for advancing research in this domain. Our contributions are as follows:

- We introduce the CME-CAD paradigm, a novel RL framework for CAD code generation. This approach integrates the complementary strengths of multiple models, fostering collaborative learning and enhancing the model's ability. 【5】
- We present CADExpert, an open-source, industrial-grade dataset that includes: (1) three-view images with precise dimension annotations, (2) code generation processes (COT) from various experts, (3) corresponding executable CADQuery code, and (4) rendered 3D CAD models.
- By aligning the problem with real-world workflows, we bridge the gap between academia and industry and conduct extensive evaluations of state-of-the-art VisionLanguage Models (VLMs) in this context.

2. Related Work

2.1. CAD Code Generation

CAD code generation aims to directly create accurate and editable 3D models from user instructions. Unlike general code generation, it is more challenging because it requires strong reasoning about space, geometry, and physical structure, while also being highly sensitive to numerical errors. Approaches like CAD-MLLM [27] and GenCAD [3] utilize vision-language models to map image inputs to CAD commands, enabling direct translation from visual representations to design instructions. Img2CAD [32] adopts a two-stage design that separates structure prediction from parameter regression, improving the flexibility of the model. CAD2Program [24] introduces a more flexible code format that enhances the expressive power of the model, facilitating more complex and richer modeling. CADCodeVerify [4] proposes an iterative verification and improvement approach for CAD code generation, refining generated code through continuous feedback loops. CAD-Llama [13] develops a hierarchical annotation pipeline that transforms parameterized 3D CAD command sequences into structured parametric CAD code. In a similar vein, CAD-Coder [9] achieves high accuracy by generating structured CadQuery code from expert-written descriptions, significantly improving the reliability and precision of generated CAD models. CAD-RL [17] is the first to successfully generate numerically accurate CADQuery code for CAD model creation.

2.2. Reinforcement Learning with Verifiable Rewards

With the development of Vision-Language Models (VLMs) [12, 15, 16, 31, 33–35], reinforcement learning with verifiable rewards (RLVR) [11, 23, 25] has demonstrated its effectiveness in facilitating complex reasoning tasks. However, despite significant improvements in performance through test-time computational scaling, their experiments reveal that the model's performance is still constrained by its inherent knowledge. Recent works by [8, 18, 30, 36] highlight that introducing structured Chain-ofThought (CoT) reasoning paths can significantly enhance the model's reasoning capabilities. Additionally, [22, 26, 29] have introduced novel training paradigms designed specifically to improve reasoning. Nevertheless, recent studies [21, 37] indicate that the on-policy learning nature of RLVR encounters difficulties in exploring the reasoning space. This limitation arises because RLVR typically biases the model toward actions more likely to yield rewards, rather than encouraging the exploration of novel knowledge or new reasoning paths. As a result, these methods tend to rely on familiar reasoning patterns rather than discovering new, knowledge-based reasoning approaches. To address this challenge, we propose the heterogeneous Collaborative Multi-Expert Reinforcement Learning (CME-CAD) paradigm, which leverages the diverse reasoning paths generated by heterogeneous pre-trained models. This approach not only overcomes cognitive constraints but also preserves the self-driven exploration capabilities of the model, enabling more robust and innovative reasoning.

3. Method

In this section, we detail the proposed Heterogeneous Collaborative Multi-Expert Reinforcement Learning (CMECAD) framework. Fig. 1 illustrates the overall structure of our method. Sec. 3.1 presents the first training stage, MultiExpert Fine-Tuning (MEFT). Sec. 3.2 elaborates on the second training stage, Multi-Expert Reinforcement Learning (MERL). Sec. 3.3 introduces our newly proposed dataset, CADExpert, designed specifically for executable and editable CAD code generation.

3.1. Multi-Expert Fine-Tuning (MEFT)

Heterogeneous Multi-Expert Chain-of-Thought (CoT) Generation. To construct expert-specific samples, we begin by selecting N heterogeneous expert models, each distinguished by a unique system prompt P_n. We define the input set as I, which includes both the instructions and the 2D engineering drawings. For each pre-trained expert, we pose an instruction I_i and obtain the corresponding answer $A_i^{(n)}$ along with the expert's CoT $C_i^{(n)}$, which details the reasoning path. These outputs are then used to construct an expert-specific sample S_n, which encapsulates the distinct reasoning style and output of each expert.

$$S_n = \left(P_n, I_i, C_i^{(n)}, A_i^{(n)}\right). \tag{1}$$

Existing leading pre-trained models face significant challenges in generating CADQuery code directly from 2D drawings. This difficulty arises primarily due to the lack of relevant pre-training data and tasks during pre-train phase. As a result, the reliability of the CoT generated for such tasks is limited. To address this issue, we propose a reversetask approach. Specifically, we provide the model with orthographic projections alongside their corresponding CADQuery code. We guide the model to reason about how to derive the corresponding code from the orthographic projections, thereby generating a more reliable reasoning path.

Training Process. In this stage, the model is trained to predict the concatenated CoT $C_i^{(n)}$ and the corresponding final answer $A_i^{(n)}$, conditioned on the input I_i and system

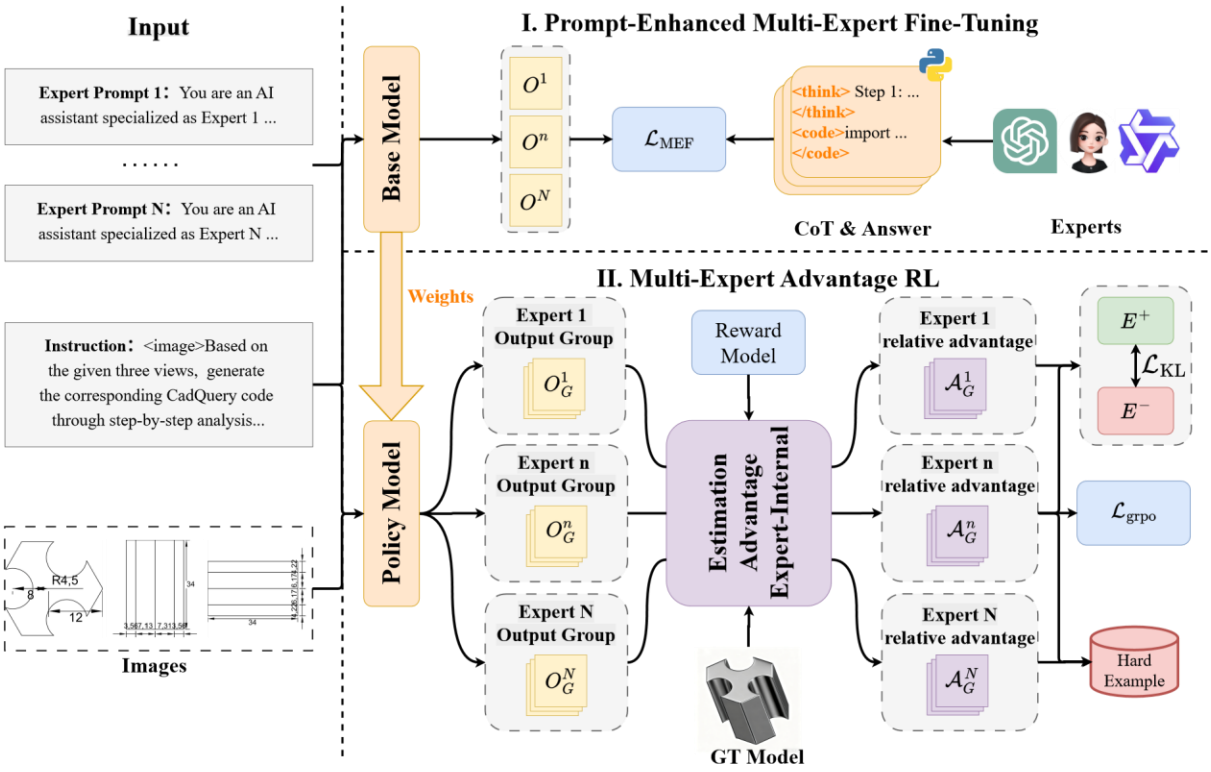

Figure 2. Overall architecture of our method CME-CAD framework. O represents the model output. O^N denotes the concatenation of the N-th expert's Chain-of-Thought (CoT) and corresponding answer. $O_G{}^N$ represents the group of G outputs generated by the N-th expert.

prompt P_n. The training objective is to maximize the likelihood of jointly generating a coherent reasoning process and a correct final answer. This is achieved by minimizing the negative log-likelihood, which encourages the model to produce outputs that align with the expert's reasoning and answer. Here, $p(\cdot \mid \cdot)$ denotes the conditional probability mass function that models the likelihood of generating the target concatenated sequence given the input and system prompt. The objective function is:

$$\mathcal{L} = -\sum_{n=1}^{N}\sum_{i=1}^{I} \log\left(p_{(\text{Concat}}(C_i^{(i)}, A_i^{(i)}) \mid P_n, I_i)\right). \quad (2)$$

3.2. Multi-Expert Reinforcement Learning (MERL)

In this section, we provide a detailed overview of the second training stage, Multi-Expert Reinforcement Learning (MERL). We focus on key components such as the Reward Function Design, Expert-Internal Advantage Estimation, Multi-Expert Collaborative Learning, and the training mechanism, Hard Negative Sample Buffering Mechanism.

Reward Function Design. To ensure that the reward function is both robust and flexible, we align the model outputs with four key objectives. First, we enforce that the output format adheres to a specific structure, combining the expert model's reasoning paths with the generated code. Second, we validate the executability of the generated code.

Third, we assess the geometric consistency of the generated 3D model with the ground truth. Finally, we ensure that the coordinate system consistency is maintained throughout the generated model.

For Format Reward R_{format}, we employ a regular expression matching mechanism to enforce the generation of a structured output by the model, where the reasoning process precedes the generated code. The reward is assigned a value of 1 if the model generates the output in the correct format; otherwise, the reward is 0.

The Executability Reward R_{exec} evaluates whether the generated CAD code is both syntactically correct and executable within a Python environment. Since the model's output is CadQuery-based Python code, we use the Python interpreter to validate its syntax and check for runtime errors. If the code parses correctly and executes without errors, it receives a reward of 1; otherwise, the reward is 0.

The Geometric Accuracy Reward R_{IoU} measures the geometric precision of the generated 3D model in

comparison to the reference model. The generated code is executed to produce the 3D model, which is then compared to the ground-truth model using the Intersection-over-Union (IoU) metric. The IoU is calculated using the Jaccard Index J:

$$R_{\text{IoU}}(M_{\text{gen}}, M_{\text{gt}}) = J(M_{\text{gen}}, M_{\text{gt}}) = \frac{|M_{\text{gen}} \cap M_{\text{gt}}|}{|M_{\text{gen}} \cup M_{\text{gt}}|}. \quad (3)$$

In 3D model reconstruction, the accuracy of the reference coordinate system is critical. During training, even if the model's reconstruction is geometrically correct, any deviation in the coordinate system can result in a complete failure of the IoU metric. To address this issue, we introduce the Work Plane Reward R_{plane}, where the uniqueness of the coordinate system is determined by both the position of the origin and the orientation of the coordinate axes. We quantify two key sources of error: the origin deviation Dis_{ori} and the normal vector deviation Dis_{vec}. The origin deviation is calculated as the Euclidean distance between the origin of the generated model and the true origin, defined as:

$$Dis_{\text{ori}} = \|O_{\text{gen}} - O_{\text{gt}}\|_2. \quad (4)$$

The normal vector deviation quantifies the misalignment of the model's coordinate axes. This is computed as:

$$Dis_{\text{vec}} = \frac{1}{2}\left[2 - \text{sim}(\boldsymbol{x}_{\text{gen}}, \boldsymbol{x}_{\text{gt}}) - \text{sim}(\boldsymbol{y}_{\text{gen}}, \boldsymbol{y}_{\text{gt}})\right]. \quad (5)$$

The overall coordinate system consistency reward R_{plane} combines the two errors as follows:

$$R_{\text{plane}} = 1 - \beta \cdot Dis_{\text{ori}} - \gamma \cdot Dis_{\text{vec}}, \quad 0 \le R_{\text{plane}} \le 1. \quad (6)$$

The total reward calculation is contingent upon satisfying both the Format Reward and the Executability Reward as essential conditions. Thus, we adopt a gating mechanism for these two components: the total reward can only be positive if both of these core conditions are met. The total reward function is defined as:

$$R = \lambda_{\text{format}} R_{\text{format}} \cdot \lambda_{\text{exec}} R_{\text{exec}} \cdot (\lambda_{\text{IoU}} R_{\text{IoU}} + \lambda_{\text{plane}} R_{\text{plane}}). \quad (7)$$

Expert-Internal Advantage Estimation. We use the model trained in the first stage as the current policy model. For each sample, it consists of system prompts P_n and the input I_i. Different system prompts P_n guide each expert's strategy, inducing the sampling of G responses for each expert. Each response is generated by the current policy model parameterized by θ. Given N experts, we generate $N \times G$ responses, where each response is denoted as (C_g^n, A_g^n). Each response (C_g^n, A_g^n) is evaluated using a reward function R, which calculates an absolute reward R_g^n for each response based on its quality:

$$R_g^n = R(C_g^n, A_g^n). \quad (8)$$

Next, we calculate the relative advantage A_n within each expert's group of responses. The relative advantage for the g-th response of expert n is computed by subtracting the average reward of all G responses from the absolute reward of the g-th response:

$$\mathcal{A}_n = R_g^n - \frac{1}{G}\sum_{g'=1}^{G} R_{g'}^n. \quad (9)$$

Thus, the relative advantage A_n quantifies how much better the g-th response is compared to the average of all responses generated by expert n. Based on the relative advantage A, we compute the GRPO [20] loss function. It is important to note that we add a non-negative truncation term $\max(A_g^n, 0)$ to avoid excessively penalizing the model's exploration ability. Given the complexity of the code generation task, truncating negative advantage values to zero ensures that the model is not immediately discouraged from exploring uncertain actions due to small negative advantages caused by minor errors in the process. The GRPO loss can be written as:

$$\mathcal{L}_{\text{GRPO}}^{(n)} = -\mathbb{E}_{A_g^n \sim \pi_\theta}\left[\log \pi_\theta(A_g^n | P_n, I_i) \cdot \max(\mathcal{A}_g^n, 0)\right]. \quad (10)$$

The term $\pi_\theta(A^n{}_g | P_n, I_i)$ represents the probability of generating the response $A^n{}_g$ by expert n, given the input I_i and system prompt P_n, under the current policy model parameterized by θ. The quantity $\mathcal{A}_g^n$ denotes the advantage for the g-th response from expert n. The expression $\max(\mathcal{A}_g^n, 0)$ is the non-negative truncation function.

Multi-Expert Collaborative Learning To facilitate knowledge exchange among experts, we introduce a multiExpert collaborative learning mechanism. For each input I_i, we compute the average absolute reward r_n for expert n across all generated responses. The expert with the highest average reward is defined as the best expert E^+, while the expert with the lowest average reward is defined as the worst expert E^-. Thus, $\log p_\theta(A_{\text{correct}} | P_{E^-}, I_i)$ represents the log-probability of a high-quality response from E^-, and $\log p_\theta(A^+ | P_{E^+}, I_i)$ represents the log-probability of a high-

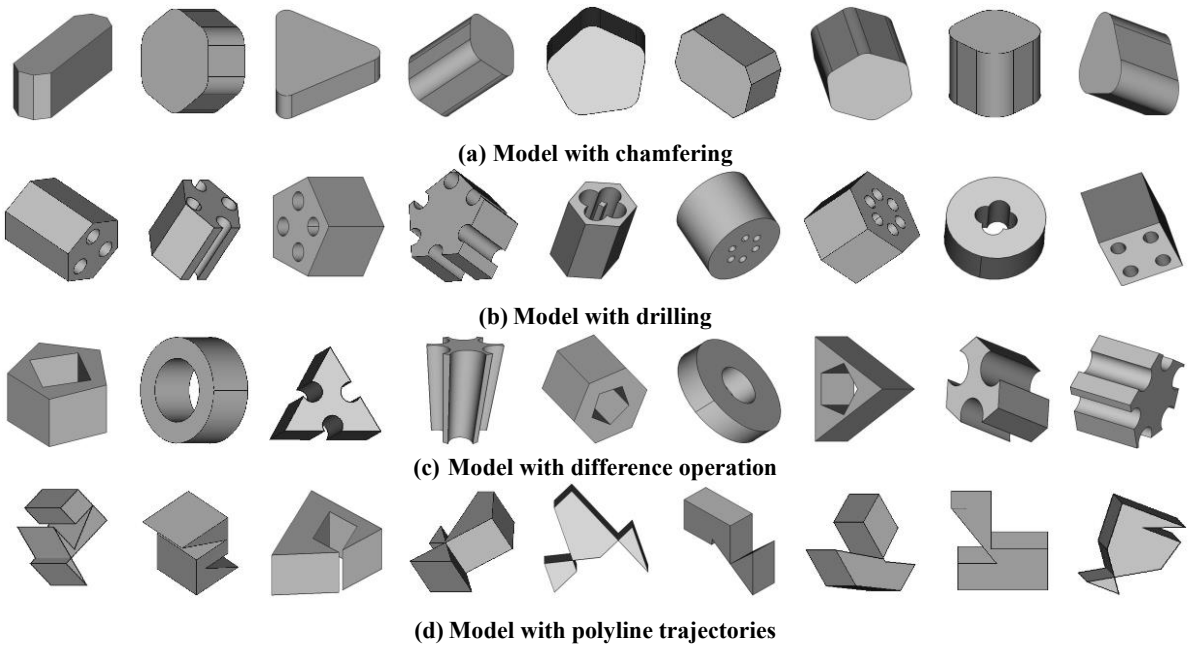

Figure 3. Examples from CADExpert, showcasing more complex industrial design challenges, including intricate features that align with real-world modeling difficulties.

quality response from E^+. To promote learning from the more effective experts, we introduce a KL divergence penalty, which forces less effective experts to learn from the better ones. We define the KL divergence penalty as follows:

$$L_{KL} = KL\left(\pi_\theta(A^+|P_{E^-}, I_i)\|\pi_\theta(A_{\text{correct}}|P_{E^+}, I_i)\right). \quad (11)$$

Thus, the KL divergence loss ensures that the worseperforming expert E^- learns from the more effective expert E^+ and gradually improves its performance. The KL divergence penalty is used to transfer knowledge from highreward solutions, without reducing the model's diversity.

This characteristic is achieved through the use of fixed and unique system prompts, which enforce differentiated reasoning styles across experts. The core objective is to guide the model to reason: *"Given your unique prompt style, how would you generate this correct answer?"* Additionally, by leveraging the vLLM inference engine for parallel batch sampling (rollouts) and the paged attention mechanism, the total training time increases by only 20%30% compared to the single-expert baseline, instead of the theoretical N-fold increase. Moreover, since vLLM pre-allocates memory based on the inference configuration, peak memory usage remains comparable. A key advantage during inference is the ability to select the best-performing expert, significantly reducing computational costs, as opposed to traditional multi-expert architectures, which require running all experts and selecting the optimal one while maintaining high-quality responses.

Hard Negative Sample Buffering Mechanism. The Hard Negative Sample Buffering Mechanism addresses the issue when all experts fail to generate a correct response for a particular query. We divide the reinforcement learning data into M parts. After training on one part (i.e., $\frac{1}{M}$ of the data), we use the $\frac{1}{M}$ portion as the test set. We maintain a buffer B that stores difficult samples. For each input I_i, if expert n generates more than K incorrect answers out of G responses, we calculate the probability of adding the sample to the buffer as $\frac{K}{G}$. Subsequently, we perform supervised fine-tuning on the data stored in the buffer B. The supervised loss for each input I_i is calculated by comparing the model's output with the ground-truth answer. Thus, the total supervised fine-tuning loss is:

$$L_{SFT} = -\sum_{B} \log p_\theta(A_{\text{correct}}|P_n, I_i). \quad (12)$$

By revisiting these challenging samples and performing targeted fine-tuning, the model can learn to better handle a broader range of input scenarios. This mechanism ensures that even the most difficult examples are not discarded, thus improving the utilization of all available data. As a result, the model's robustness is enhanced, especially when dealing with edge cases or scenarios that

are not adequately covered by simpler samples. This process promotes continuous improvement.

3.3. CADExpert

A significant bottleneck in the progress of CAD code generation lies in the lack of high-quality open-source datasets that meet industrial-grade standards. Existing datasets suffer from several key issues, including:

- Lack of Availability: Descriptions of trajectories and other detailed textual data are often not available.
- Missing Precise Dimensional Information: The input images used for CAD generation are usually sketches without precise, detailed dimensional annotations.
- Simplistic Operations: Existing datasets lack examples of complex industrial operations, such as chamfering, drilling, and difference operations.

To address these challenges, we introduce CADExpert, a benchmark dataset specifically designed for executable and editable CAD code generation. CADExpert contains 17,299 instances. As shown in Fig. 3, our model addresses several challenges not typically included in previous datasets, but which are essential for real-world applications. These challenges include operations such as chamfering, drilling, set difference, and complex trajectories, all of which reflect the intricacies encountered in industrial design tasks. Each instance in the dataset consists of: orthographic projections with precise dimension annotations, detailed reasoning paths generated by different expert models, the corresponding executable CADQuery code, and the rendered 3D CAD model.

Data Generation Process. The data generation process consists of several key steps:

- Random Generation of CADQuery Code with Constraints: On random base planes (e.g., XY, YZ, XZ), we design multiple basic shape trajectories (e.g., rectangles, regular polygons, circles, polyline trajectories). These shapes are then extruded along a vertical plane and subjected to various complex operations, such as drilling, chamfering, and difference operations. The resulting CADQuery code is executable, editable, and randomly generated with specific constraints.
- CADQuery Code Filtering: After generating the random CADQuery code, we implement a comprehensive two-stage filtering process to ensure both the validity and functional integrity of the code. In the first stage, executable validation is performed by running the generated code through a Python interpreter to check for syntax errors, runtime exceptions, and logical consistency. In the second stage, the manual review process is initiated. A team of 10 experts is responsible for performing a sampling-based review of the generated code. The reviewers visually inspect the generated CAD models to verify that they align with the intended design features, focusing on geometric accuracy and the correct implementation of the code's functionality. In cases where ambiguities or uncertainties arise, the reviewers collaborate to reach a consensus. For particularly complex models, we implement a multi-round validation mechanism. In this process, the reviewers may iterate on the code generation process and provide feedback to refine the code, ensuring that the final output meets the required quality standards and adheres to the intended design specifications.
- Generating Annotated Images: Based on the generated CADQuery code, we execute the code to produce a STEP file. Using the FreeCAD [19] API, we convert this STEP file into orthogonal projections and generate accurate 2D DXF files. These DXF files are then annotated with dimension lines using the ezdxf library, and the final DXF files are rendered into raster images.
- Expert CoT Process Generation: Using multiple expert models, we input the generated code and corresponding orthogonal projections to produce varying reasoning paths. Each expert generates a unique reasoning path for solving the problem, contributing to the diversity of the dataset.

This detailed process aims to resolve the major limitations of existing datasets by providing more accurate, complex, and industry-relevant data, thus facilitating more effective and robust CAD code generation models.

4. Experimental Results

4.1. Implementation Details

The base model used for this experiment is Qwen3-VL-4BInstruct [28], and the training is conducted on a setup with 8 H100 GPUs. During the Multi-Expert Fine-Tuning stage, the learning rate is set to 1×10^{-5}, with a batch size of 32. In the Multi-expert rainforcement learning stage, the learning rate remains at 1×10^{-5}, with a batch size of 8. For all RL-trained models, we maintain 4 rollouts per prompt. The temperature is set to 0.9 to control the randomness of predictions during training. We utilize three expert models in our setup. Expert 1 is Qwen3–VL-Plus [28], Expert 2 is

Table 1. The comparison on CADExpert is presented in two sections. The upper block reports results using pretrained SOTA VLMs without any fine-tuning, while the lower block displays results after fine-tuning. $*$ denote our re-implementation trained on the same benchmark.

Model	IOU(%)↑	Mean CD↓	Med CD↓	Exec.(%)↑

LLaVA-1.5 [14]	0.73	29.36	8.14	4.79	Phi-3.5-Vision [1]	3.44	27.92	7.75
8.50								
InternVL2.5 [6]	11.98	22.27	7.36	23.92	Qwen2.5-VL [5]	19.21	20.68	6.64
31.27								
InternVL3 [38]	24.59	21.40	6.62	29.68	Gemini2.5 Pro [7]	30.86	14.13	5.97
39.40								
GPT5-Mini [2]	35.15	9.89	4.81	47.13	Doubao-1.6 [10]	35.62	7.54	4.35
52.89								
Qwen3-VL [28]	37.04	6.96	3.84	54.79				
CAD-RL∗ [17]	71.84	1.38	0.36	97.32				
Ours	80.71	1.00	0.11	98.25				

GPT-5-Mini [2], Expert 3 is Doubao-Seed-1.6-Vision [10]. For inference, to ensure consistent results and eliminate the impact of randomness, no sampling methods (such as temperature sampling or beam search) are employed during the testing phase for any of the models.

4.2. Evaluation Metrics

We evaluate the performance of Vision-Language Models (VLMs) and our method on CADExpert using four complementary metrics that focus on geometric accuracy, code executability, and structural precision. These metrics include: (1) Intersection-over-Union (IoU): This metric measures the volumetric overlap between the predicted and reference shapes. Given its strict spatial alignment requirement, IoU is particularly effective for assessing the accuracy of shapes in industrial CAD applications. Higher value is better for IoU. (2) Mean Chamfer Distance (Mean CD): This metric calculates the average geometric discrepancy between the generated and ground-truth models in point cloud space. It provides an overall measure of shape fidelity. (3) Median Chamfer Distance (Med CD): This metric captures the typical error per sample, offering a more robust measure by being less sensitive to outliers. It provides a stable estimate of model performance. Lower values are preferred for CD. (4) Executability (Exec.): This metric directly evaluates the functional validity of the generated code, assessing whether the generated CAD model can be executed correctly. Higher value is better for executability.

4.3. Main Results

Benchmarking CAD Code Generation. Tab. 1 presents a comprehensive evaluation on CADExpert. The comparison includes nine state-of-the-art pretrained vision-language models (VLMs), all evaluated without any task-specific post training. We also benchmark two fine-tuned baselines: CAD-RL [17] and our method CME-CAD. As illustrated in Fig. 3, our dataset significantly outperforms existing opensource datasets in terms of complexity. A detailed comparison can be found in the supplementary materials. In this challenging context, our method achieves comprehensive superiority across all evaluation metrics. Specifically, we attain an IoU of 80.71%, representing a performance improvement of 8.87% points over the current SOTA methods. This demonstrates the effectiveness of our approach and its ability to handle more complex and realistic CAD design tasks. These improvements are also attributed to a significant increase in code executability, with our method achieving a code executability rate of 98.25%. Such performance is highly valuable in real-world applications, as it means engineers

Table 2. Results of Multi-Expert learning compared to individual Expert learning.The table shows four sections: performance of a single expert with SFT, performance with multi-expert data in SFT, performance of a single expert with both SFT and GRPO, and the final results of our approach.

Model	IOU(%)↑	Mean CD↓	Med CD↓	Exec.(%)↑
Expert1-SFT	44.02	5.36	3.04	85.62
Expert2-SFT	38.08	7.05	3.74	82.14
~~Expert3-SFT~~	~~40.91~~	~~5.34~~	~~3.27~~	~~85.35~~
MoE-SFT (Expert1)	57.19	2.99	0.73	95.12
MoE-SFT (Expert2)	61.30	2.23	0.63	97.06
MoE-SFT (Expert3)	64.45	1.63	0.42	96.99
E1-SFT-GRPO	53.42	3.97	0.98	95.63
~~E2-SFT-GRPO~~	~~49.29~~	~~4.74~~	~~1.51~~	~~93.96~~
E3-SFT-GRPO	52.38	4.70	1.23	96.88
Ours (E1)	75.69	1.28	0.28	98.35
~~Ours (E2)~~	~~75.06~~	~~1.21~~	~~0.23~~	~~97.18~~
Ours (E3)	80.71	1.00	0.14	98.25

Table 3. Ablation studies on each component of CME-CAD, with results reported for the best-performing experts for brevity. “EIAE” refers to Expert-Internal Advantage Estimation, “HSB” stands for Hard Negative Sample Buffering Mechanism, and “MECL” denotes Multi-Expert Collaborative Learning.

EIAE	HSB	MECL	IOU(%)↑	Mean CD↓	Exec.(%)↑
✗	✗	✗	64.45	1.63	96.99
✓	✗	✗	73.89	1.33	98.28
✓	✓	✗	78.14	1.13	98.22
✓	✗	✓	76.71	1.28	98.19
✓	✓	✓	80.71	1.00	98.25

can rely heavily on the model's outputs, needing only to make minimal modifications to a small subset of more complex samples.

Ablation Study for Multi-Expert Learning and Individual Expert learning. Tab. 2 illustrates the performance differences between multi-expert learning and single-expert learning. As shown in the table, compared to using a single expert for SFT, multi-expert training leads to improvements across all evaluation metrics for each expert model. This demonstrates the successful facilitation of collaborative learning among the models. Despite completing the full SFT + GRPO process, a single expert model faces a performance ceiling due to the inherent limitations of its reasoning path. By incorporating data from multiple experts, the model can overcome some of these limitations with just SFT. Ultimately, applying our complete methodology results in significant improvements, surpassing the performance achieved by a single expert.

Ablation Studies on Components of CME-CAD. Tab. 3 demonstrates the impact of each individual component of CME-CAD on the model's performance. Notably, the introduction of Expert-Internal Advantage Estimation leads to significant improvements in code executability. This is primarily due to its role in helping the model select the most suitable expert for each task. The Hard Negative Sample Buffering Mechanism has the most substantial effect on overall performance, which can be attributed to the high complexity of the CADExpert dataset. By continuously increasing the utilization of difficult samples during training, the model's performance is significantly enhanced. Furthermore, the joint use of Expert-Internal Advantage Estimation and Multi-Expert Collaborative Learning fully activates the potential of our training framework, enabling the model to effectively collaborative learning across different heterogeneous experts.

5. Conclusion

In this work, we introduce the CME-CAD paradigm, which leverages the complementary strengths of heterogeneous expert models to address key challenges in the generation of executable and editable CAD code. Our method effectively bridges the gap between the complexities of industrial design workflows and the capabilities of machine learning models, offering a novel solution to automate the creation of precise CAD models from 2D drawings input. By combining Multi-Expert Fine-Tuning (MEFT) and MultiExpert Reinforcement Learning (MERL), we enable the model to explore diverse reasoning paths and improve performance, particularly in complex design scenarios where traditional methods fall short. Furthermore, the introduction of CADExpert provides a high-quality benchmark that facilitates the development and evaluation of CAD code generation methods. This dataset, specifically designed to meet industrial-grade requirements, enables more robust training and evaluation. Overall, this work lays the foundation for more efficient, accurate, and editable CAD model generation.

6. Acknowledgement

This work was supported by the National Natural Science Foundation of China (NSFC) General Program (Grant No. 62576110) and the Shanghai Municipal Special Program for Promoting High-Quality Industrial Development in 2025 – Innovative Development of Pioneer Industries (Artificial Intelligence Track) (No. 2025-GZL-RGZN02034). The authors are with the Fudan University–China Railway 24th Bureau Joint Laboratory for Intelligent Construction Engineering Technologies on Operational Railway Lines.

References

[1] Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, et al. Phi-3 technical report: A highly capable language model locally on your phone. *arXiv preprint arXiv:2404.14219*, 2024. 7

[2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 7

[3] Md Ferdous Alam and Faez Ahmed. Gencad: Imageconditioned computer-aided design generation with transformer-based contrastive representation and diffusion priors. *arXiv preprint arXiv:2409.16294*, 2024. 2

[4] Kamel Alrashedy, Pradyumna Tambwekar, Zulfiqar Zaidi, Megan Langwasser, Wei Xu, and Matthew Gombolay. Generating cad code with vision-language models for 3d designs. *arXiv preprint arXiv:2410.05340*, 2024. 2

[5] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yuanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report, 2025. 7

[6] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of*

the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 24185–24198, 2024. 7

[7] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025. 7

[8] Mehdi Fatemi, Banafsheh Rafiee, Mingjie Tang, and Kartik Talamadupula. Concise reasoning via reinforcement learning. *arXiv preprint arXiv:2504.05185*, 2025. 3

[9] Yandong Guan, Xilin Wang, Xingxi Ming, Jing Zhang, Dong Xu, and Qian Yu. Cad-coder: Text-to-cad generation with chain-of-thought and geometric reward. *arXiv preprint arXiv:2505.19713*, 2025. 3

[10] Dong Guo, Faming Wu, Feida Zhu, Fuxing Leng, Guang Shi, Haobin Chen, Haoqi Fan, Jian Wang, Jianyu Jiang, Jiawei Wang, et al. Seed1. 5-vl technical report. *arXiv preprint arXiv:2505.07062*, 2025. 7

[11] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35:22199–22213, 2022. 3

[12] Zhenglun Kong, Yize Li, Fanhu Zeng, Lei Xin, Shvat Messica, Xue Lin, Pu Zhao, Manolis Kellis, Hao Tang, and Marinka Zitnik. Token reduction should go beyond efficiency in generative models–from vision, language to multimodality. *arXiv preprint arXiv:2505.18227*, 2025. 3

[13] Jiahao Li, Weijian Ma, Xueyang Li, Yunzhong Lou, Guichun Zhou, and Xiangdong Zhou. Cad-llama: leveraging large language models for computer-aided design parametric 3d model generation. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 18563–18573, 2025. 2

[14] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 26296–26306, 2024. 7

[15] Ke Niu, Haiyang Yu, Yuwen Chen, Teng Fu, Mengyang Zhao, Bin Li, Xiangyang Xue, et al. Creft-cad: Boosting orthographic projection reasoning for cad via reinforcement fine-tuning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 3

[16] Ke Niu, Haiyang Yu, Mengyang Zhao, Teng Fu, Siyang Yi, Wei Lu, Bin Li, Xuelin Qian, and Xiangyang Xue. Chatreid: Open-ended interactive person retrieval via hierarchical progressive tuning for vision language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 24245–24254, 2025. 3

[17] Ke Niu, Haiyang Yu, Zhuofan Chen, Mengyang Zhao, Teng Fu, Bin Li, and Xiangyang Xue. From intent to execution: Multimodal chain-of-thought reinforcement learning for precise cad code generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8160–8167, 2026. 3, 7, 8

[18] Bo Pang, Hanze Dong, Jiacheng Xu, Silvio Savarese, Yingbo Zhou, and Caiming Xiong. Bolt: Bootstrap long chainof-thought in language models without distillation. *arXiv preprint arXiv:2502.03860*, 2025. 3

[19] Juergen Riegel, Werner Mayer, and Yorik van Havre. Freecad. *Freecadspec2002*, 2016. 7

[20] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. 5

[21] Parshin Shojaee, Iman Mirzadeh, Keivan Alizadeh, Maxwell Horton, Samy Bengio, and Mehrdad Farajtabar. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity. *arXiv preprint arXiv:2506.06941*, 2025. 3

[22] Xuerui Su, Shufang Xie, Guoqing Liu, Yingce Xia, Renqian Luo, Peiran Jin, Zhiming Ma, Yue Wang, Zun Wang, and Yuting Liu. Trust region preference approximation: A simple and stable reinforcement learning algorithm for llm reasoning. *arXiv preprint arXiv:2504.04524*, 2025. 3

[23] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022. 3

[24] Xilin Wang, Jia Zheng, Yuanchao Hu, Hao Zhu, Qian Yu, and Zihan Zhou. From 2d cad drawings to 3d parametric models: A vision-language approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 7961– 7969, 2025. 2

[25] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022. 3

[26] Liang Wen, Yunke Cai, Fenrui Xiao, Xin He, Qi An, Zhenyu Duan, Yimin Du, Junchen Liu, Tanglifu Tanglifu, Xiaowei Lv, et al. Light-r1: Curriculum sft, dpo and rl for long cot from scratch and beyond. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, pages 318–327, 2025. 3

[27] Jingwei Xu, Zibo Zhao, Chenyu Wang, Wen Liu, Yi Ma, and Shenghua Gao. Cad-mllm: Unifying multimodalityconditioned cad generation with mllm. *arXiv preprint arXiv:2411.04954*, 2024. 2

[28] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. 7

[29] Wang Yang, Hongye Jin, Jingfeng Yang, Vipin Chaudhary, and Xiaotian Han. Thinking preference optimization. *arXiv preprint arXiv:2502.13173*, 2025. 3

[30] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning. *arXiv preprint arXiv:2502.03387*, 2025. 3

[31] Weijie Yin, Yongjie Ye, Fangxun Shu, Yue Liao, Zijian Kang, Hongyuan Dong, Haiyang Yu, Dingkang Yang, Jiacong Wang, Han Wang, et al. Sail-vl2 technical report. *arXiv preprint arXiv:2509.14033*, 2025. 3

[32] Yang You, Mikaela Angelina Uy, Jiaqi Han, Rahul Thomas, Haotong Zhang, Suya You, and Leonidas Guibas. Img2cad: Reverse engineering 3d cad models from images through vlm-assisted conditional factorization. *arXiv preprint arXiv:2408.01437*, 2024. 2

[33] Haiyang Yu, Jinghui Lu, Yanjie Wang, Yang Li, Han Wang, Can Huang, and Bin Li. Eve: Towards end-to-end video subtitle extraction with vision-language models. *arXiv preprint arXiv:2503.04058*, 2025. 3

[34] Haiyang Yu, Yuchuan Wu, Fan Shi, Lei Liao, Jinghui Lu, Xiaodong Ge, Han Wang, Minghan Zhuo, Xuecheng Wu, Xiang Fei, et al. Benchmarking vision-language models on chinese ancient documents: From ocr to knowledge reasoning. *arXiv preprint arXiv:2509.09731*, 2025.

[35] Haiyang Yu, Siyang Yi, Ke Niu, Minghan Zhuo, and Bin Li. Umit: Unifying medical imaging tasks via vision-language models. *arXiv preprint arXiv:2503.15892*, 2025. 3

[36] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, and Gao Huang. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025. 3

[37] Rosie Zhao, Alexandru Meterez, Sham Kakade, Cengiz Pehlevan, Samy Jelassi, and Eran Malach. Echo chamber: Rl post-training amplifies behaviors learned in pretraining. *arXiv preprint arXiv:2504.07912*, 2025. 3

[38] Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao, Zhangwei Gao, Erfei Cui, Xuehui Wang, Yue Cao, Yangzhou Liu, Xingguang Wei, Hongjie Zhang, Haomin Wang, Weiye Xu, Hao Li, Jiahao Wang, Nianchen Deng, Songze Li, Yinan He, Tan Jiang, Jiapeng Luo, Yi Wang, Conghui He, Botian Shi, Xingcheng Zhang, Wenqi Shao, Junjun He, Yingtong Xiong, Wenwen Qu, Peng Sun, Penglong Jiao, Han Lv, Lijun Wu, Kaipeng Zhang, Huipeng Deng, Jiaye Ge, Kai Chen, Limin Wang, Min Dou, Lewei Lu, Xizhou Zhu, Tong Lu, Dahua Lin, Yu Qiao, Jifeng Dai, and Wenhai Wang. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models, 2025. 7